

Kafka Streams Windowing - Complete Student Guide

A beginner-friendly guide to understand Tumbling, Hopping, Count-Based, and Session Windows.

1. What is Windowing?

Streams are unbounded because data keeps arriving continuously. Windowing divides an infinite stream into finite chunks called windows so that computations such as counts, sums, averages, and aggregations can be performed.

2. Why Do We Need Windowing?

Suppose an e-commerce site receives orders all day long. Questions such as 'How many orders arrived in the last 5 minutes?' or 'What were total sales during the last hour?' require grouping records. Windowing provides that grouping.

3. Tumbling Windows

Tumbling windows are fixed-size, non-overlapping windows. Each event belongs to exactly one window.

Example:

Window size = 5 minutes

0-5, 5-10, 10-15.

Use cases: sales per hour, website visits per minute, sensor readings every 10 seconds.

Advantages: simple, efficient, easy to understand.

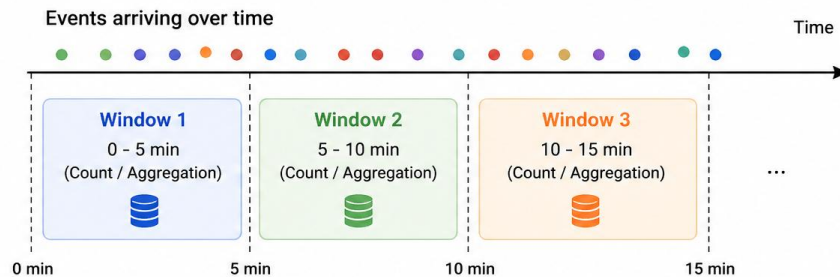
Disadvantages: no rolling view of data.

Tumbling Windows

Tumbling windows are fixed-size, non-overlapping windows. Each event belongs to **exactly one window**.

Example:

- Window size = 5 minutes
- Windows: 0-5, 5-10, 10-15, ...



Use cases



Sales per hour



Website visits per minute



Sensor readings every 10 seconds



Advantages



Simple



Efficient



Easy to understand



Disadvantages



No rolling view of data

4. Hopping Windows

Hopping windows overlap. They are defined using a window size and an advance interval.

Example:

Window size = 1 hour

Advance interval = 15 minutes

Windows:

00:00-01:00

00:15-01:15

00:30-01:30

00:45-01:45

The same event may appear in multiple windows.

Use cases: trend analysis, rolling sales reports, monitoring dashboards.

Advantages: continuous insights.

Disadvantages: more storage and processing overhead.

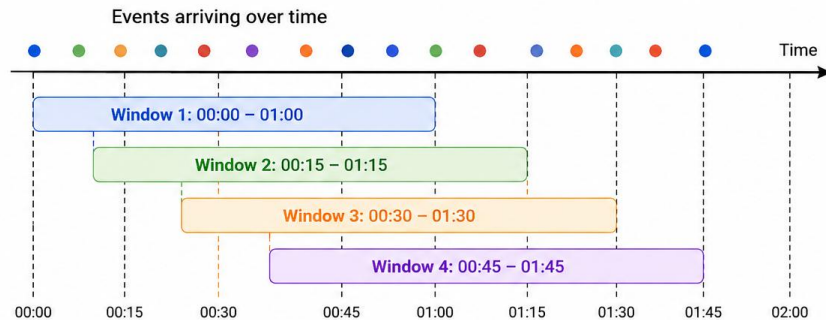
Hopping Windows

Hopping windows **overlap**. They are defined using a window size and an **advance interval**.

Example:

- Window size = 1 hour
- Advance interval = 15 minutes
- Windows:
 - 00:00 – 01:00
 - 00:15 – 01:15
 - 00:30 – 01:30
 - 00:45 – 01:45

The same event may appear in multiple windows.



Use cases

- Trend analysis
- Rolling sales reports
- Monitoring dashboards



Advantages

- Continuous insights



Disadvantages

- More storage and processing overhead

5. Tumbling vs Hopping

Tumbling gives one result per completed window.

Hopping gives rolling results at regular intervals.

If sales drop at 10:20 AM, a tumbling 1-hour window may reveal it at 11:00 AM. A hopping window with a 15-minute advance interval can expose the trend much earlier.

6. Count-Based Windows

Count windows are based on the number of events rather than time.

Example:

Process every 10 purchases.

Events 1-10 form one window, 11-20 form the next.

Use cases: transaction batches, sensor measurements, log processing.

7. Session Windows

Session windows group events based on user activity. A session remains open while events continue arriving. If inactivity exceeds a threshold, a new session starts.

Example:

Session timeout = 30 minutes.

Clicks at 10:00, 10:05, 10:20 belong to one session.

A click at 11:00 starts a new session.

Use cases: website analytics, user behavior analysis, shopping carts.

8. Kafka Streams Examples

Tumbling:

```
TimeWindows.ofSizeWithNoGrace(Duration.ofMinutes(5))
```

Hopping:

```
TimeWindows.ofSizeAndGrace(Duration.ofHours(1), Duration.ofMinutes(5))  
    .advanceBy(Duration.ofMinutes(15))
```

Session:

```
SessionWindows.ofInactivityGapWithNoGrace(Duration.ofMinutes(30))
```

9. Summary

Tumbling = Fixed size, no overlap.

Hopping = Fixed size, overlap.

Count Window = Based on event count.

Session Window = Based on inactivity gap.

Rule of thumb:

Periodic reports -> Tumbling

Rolling analytics -> Hopping

Fixed number of records -> Count Window

User activity tracking -> Session Window